

DPoP (Demonstrating Proof-of-Possession) Support

Added in v2.9.0 | Automatic detection and handling across all tools Implements RFC 9449 — OAuth 2.0 Demonstrating Proof-of-Possession

What is DPoP? (For Everyone)

DPoP is a security mechanism that prevents stolen access tokens from being reused by attackers. Think of it like this:

- **Without DPoP:** An access token is like a house key — anyone who has it can open the door.
- **With DPoP:** An access token is like a house key that only works when YOU hold it — if someone steals the key, it won't work for them because the lock also checks your fingerprint.

How It Works (Simple Version)

1. Your browser generates a unique key pair (like a fingerprint) when you log in.
2. Every API request includes a "proof" — a signed message that says "I am the original owner of this token, and I'm sending this specific request right now."
3. The server verifies that the proof matches the token and the request. If someone intercepts the token, they can't create valid proofs because they don't have the private key.

How to Identify DPoP in Your Application

Look for these headers in your browser's Developer Tools (Network tab):

Header	When It Appears	What It Means
<code>dpop-pf</code>	On the login request only	"Pre-flight" proof — proves key ownership before getting a token
<code>dpop</code>	On every authenticated request	Proof that you own the token and are making this specific request

Authorization: Bearer <token>	On every authenticated request	The actual access token (always paired with <code>dpop</code>)
----------------------------------	---	--

If you see `dpop` or `dpop-pf` headers in your recording, your application uses DPOP.

DPoP Token Structure

A DPOP proof looks like a JWT (three base64-encoded parts separated by dots):

```
eyJhbGciOiJFUzI1NiIsInRScCI6ImRwb3Arand0In0.eyJodHUiOiJodHRwczovL2FwaS5leGFtcGxlL
```

Decoded, it contains:

- **Header:** `{ "alg": "ES256", "typ": "dpop+jwt", "jwk": { ... } }` — the algorithm and public key
- **Payload:** `{ "htu": "https://api.example.com/data", "htm": "GET", "iat": 1700000000, "jti": "unique-id" }` — the target URL, method, timestamp, and unique ID
- **Signature:** ECDSA P-256 digital signature

Key Rules

1. Every request gets a unique proof — you cannot reuse a DPOP proof across requests
2. The `htu` must match the request **URL** — a proof for `/api/accounts` won't work for `/api/transfers`
3. The `htm` must match the HTTP **method** — a proof for `GET` won't work for `POST`
4. The `ath` claim (on `dpop` headers) is a SHA-256 hash of the access token — binds the proof to the specific token
5. The `dpop-pf` header uses a different `htu` — it points to the OAuth token endpoint (e.g., `PingFederate`), not the login URL

How LRE Toolkit Handles DPOP

Automatic Detection

The toolkit automatically detects DPop in **all** input sources:

Input Source	Detection Method
Postman/Bruno collection	Scans pre-request scripts for DPoP code patterns (jose library, dpop+jwt type)
Postman/Bruno collection	Scans request headers for dpop or dpop-pf header names
JMeter .jmx file	Scans JSR223/BeanShell scripts for DPoP patterns
JMeter .jmx file	Scans request headers for dpop or dpop-pf header names
HAR file (Recorder)	Scans recorded request headers for dpop or dpop-pf
HAR file (Script Studio)	Same as Recorder, plus correlates the Bearer token automatically

What Gets Generated

DevWeb (JavaScript) Scripts

```
// Module level - loaded once
const { getDpopProof } = require('./dpop-helper.js');

// Initialize - key pair generated once per VUser
load.global.dpop_jwk = null;

// Before EACH request that needs DPoP:

// Login request (dpop-pf) - no access token yet, htu = token endpoint
load.global.dpop_pf_proof = getDpopProof(
    'https://auth-server.example.com/as/token.oauth2', // htu from token endpoint
    'POST', // htm
    load.global.dpop_jwk // key pair (auto-generated)
);

// Authenticated request (dpop) - includes access token hash
load.global.dpop_proof = getDpopProof(
    'https://api.example.com/accounts', // htu = request URL
    'GET', // htm = request method
    load.global.dpop_jwk, // same key pair
    load.global.AccessToken // access token for ath claim
);
```

VuGen Web HTTP/HTML (C) Scripts

```
// Before EACH request that needs DPoP:
web_js_run(
    "Code=initDpopKey(LR.getParam('dpop_jwk')); "
    "var proof = generateDpopProof("
    "'https://api.example.com/accounts', 'GET', "
    "LR.getParam('AccessToken')); proof;",
    "ResultParam=dpop_proof",
    SOURCES,
    "File=jsrsasign-vugen.js", ENDITEM,
    "File=dpop.js", ENDITEM,
    LAST);

// Then the header and request:
web_add_header("Dpop", "{dpop_proof}");
web_add_header("Authorization", "Bearer {AccessToken}");
web_url("accounts", "URL=https://...", ...);
```

Generated Files

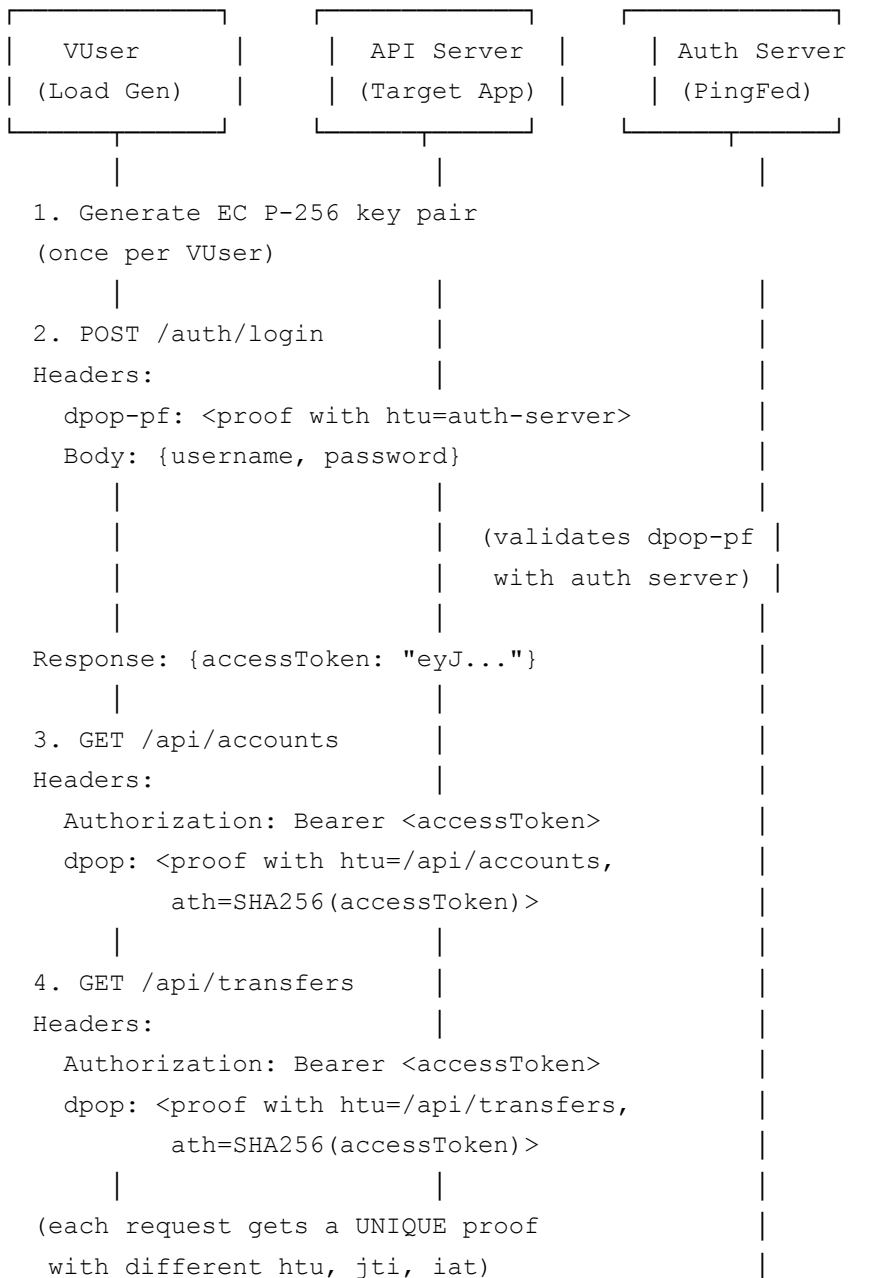
File	Protocol	Purpose
dpop-helper.js	DevWeb	EC P-256 key generation + DPoP proof signing (Node.js crypto)
dpop.js	VuGen	DPoP proof generation using jsrsasign library
jsrsasign-vugen.js	VuGen	Crypto library for ECDSA signing (exports-stripped for VuGen JS engine)

Runtime Settings

When DPoP is detected, the toolkit automatically configures:

- DevWeb: No special settings needed (Node.js crypto is built-in)
- VuGen: `EnableJsForTransport=1` in `default.cfg` (enables the JavaScript engine for `web_js_run`)

DPoP Flow Diagram



Technical Details

dpop-helper.js (DevWeb)

Used by DevWeb scripts running on Node.js. Uses built-in `crypto` module for:

- Key generation: `crypto.generateKeyPairSync('ec', { namedCurve: 'prime256v1' })`
- Signing: `crypto.sign('sha256', data, privateKey)` with DER→raw R|S conversion
- Token hashing: `crypto.createHash('sha256').update(token).digest('base64url')`

API:

```
// Generate a new EC P-256 key pair
const jwk = generateEcP256KeyPair();
// Returns: { kty: 'EC', crv: 'P-256', x: '...', y: '...', d: '...' }

// Generate a DPoP proof
const proof = getDpopProof(htu, htm, jwk, accessToken);
// htu: target URL (string)
// htm: HTTP method (string)
// jwk: private key (object, JSON string, or null to auto-generate)
// accessToken: optional Bearer token for ath claim (string)
// Returns: signed JWT string (header.payload.signature)
```

ECDSA Signature Format: Node.js `crypto.sign()` produces DER-encoded signatures (~70-72 bytes). JWT/JWS (RFC 7515) requires raw R|S format (exactly 64 bytes for P-256). The `derToRawEcDSA()` function handles this conversion.

dpop.js (VuGen)

Used by VuGen Web HTTP/HTML scripts via `web_js_run`. Uses **jsrsasign** library for:

- Key generation: `KEYUTIL.generateKeypair("EC", "P-256")`
- Signing: `KJUR.jws.JWS.sign("ES256", header, payload, privateKey)` — produces correct raw R|S format
- Token hashing: `KJUR.crypto.MessageDigest({alg: "sha256"})` for ath claim

API:

```
// Initialize key pair (call once, or pass existing JWK)
initDpopKey(jwkJsonString);

// Generate proof (call before each request)
var proof = generateDpopProof(htu, htm, accessToken);
```

jsrsasign-vugen.js

A patched version of jsrsasign (v10.8.6) with `exports.*` lines removed. VuGen's `web_js_run` JavaScript engine does not support CommonJS modules (`exports`, `module.exports`, `require`). The patched version exposes `KJUR`, `KEYUTIL`, etc. as global variables.

Important: This file is ONLY used for DPoP. JWT signing continues to use the original

`jsrsassign.js` which has the `createJWT` function appended.

VuGen Script Ordering Rules

The correct order of VuGen C function calls for DPoP requests:

1. `web_js_run()` ← Generate DPoP proof (clears pending registrations)
2. `web_reg_save_param_*`() ← Register correlation extraction (pending)
3. `web_add_header()` ← Set all headers (DPoP, Authorization, etc.)
4. `web_url / web_custom_request` ← HTTP request (triggers extraction)

Critical: `web_js_run` clears any pending `web_reg_save_param` registrations. It must come BEFORE correlation registrations, not after.

VuGen Function Syntax Reference

```
// JSON extraction - NO Ord= argument allowed
web_reg_save_param_json("ParamName=AccessToken",
    "QueryString=$.data.accessToken",
    LAST);

// Boundary extraction - Ord= IS allowed
web_reg_save_param("ParamName=csrf_token",
    "LB=csrf_token=",
    "RB=;",
    "Search=Headers",
    "Ord=1",
    LAST);

// DPoP proof generation
web_js_run(
    "Code=initDpopKey(LR.getParam('dpop_jwk')); "
    "var proof = generateDpopProof('https://...', 'GET', "
    "LR.getParam('AccessToken')); proof;",
    "ResultParam=dpop_proof",
    SOURCES,
    "File=jsrsassign-vugen.js", ENDITEM,
    "File=dpop.js", ENDITEM,
    LAST);
```

Troubleshooting

Common Errors

Error	Cause	Fix
<code>JWK</code> must be an object or JSON string	DevWeb: key generation failed	Fixed in v2.9.0 — uses <code>KeyObject</code> directly
<code>exports</code> is not defined	VuGen: wrong <code>jsrsasign</code> build	Use <code>jsrsasign-vugen.js</code> (exports stripped)
DPoP key pair not initialized	VuGen: <code>initDpopKey</code> not called	Ensure <code>web_js_run</code> loads both <code>jsrsasign-vugen.js</code> AND <code>dpop.js</code>
Parameter <code>{AccessToken}</code> not found	VuGen: <code>web_js_run</code> cleared pending <code>web_reg_save_param</code>	Move <code>web_js_run</code> BEFORE <code>web_reg_save_param</code>
HTTP 400 Bad Request	<code>htu</code> contains <code>{ServerHost}</code> instead of real hostname	Fixed in v2.9.0 — uses raw URL in <code>web_js_run</code> Code
HTTP 403 Forbidden	Missing <code>ath</code> claim in DPoP proof	Fixed in v2.9.0 — passes access token for SHA-256 hash
HTTP 500 Server Error	Invalid ECDSA signature format (DER instead of raw R S)	Fixed in v2.9.0 — <code>derToRawEcdsa()</code> conversion
<code>Ord=1</code> unrecognized	<code>web_reg_save_param_json</code> doesn't support <code>Ord</code>	Fixed in v2.9.0 — removed from all json calls
<code>EnableJsForTransport</code> error	JS engine not enabled in runtime settings	Fixed in v2.9.0 — auto-set to 1 when DPoP detected

How to Verify DPoP is Working

1. Check the VUser log for `web_js_run` was successful — the proof was generated
2. Check the response — HTTP 200 means the server accepted the DPoP proof
3. Decode the proof (optional) — paste the `_dpop_proof` value into jwt.io to inspect the claims

Supported Scenarios

Scenario	DevWeb	VuGen	Notes
DPoP from Postman collection (with script)	✓	✓	Detected from pre-request script
DPoP from Postman collection (headers only)	✓	✓	Detected from header names
DPoP from Bruno collection	✓	✓	Same as Postman
DPoP from JMeter .jmx	✓	✓	Detected from scripts or headers
DPoP from HAR (Recorder)	✓	✓	Detected from recorded headers
DPoP from HAR (Script Studio)	✓	✓	Detected + correlated automatically
<code>dpop</code> header (authenticated requests)	✓	✓	With <code>ath</code> claim
<code>dpop-pf</code> header (login/pre-flight)	✓	✓	<code>htu</code> extracted from recorded token
Multiple DPoP headers per request	✓	✓	Separate proof for each
Bearer token correlation	✓	✓	Generic detection, not hardcoded
EC P-256 key reuse across requests	✓	✓	Same key pair for entire VUser session

For Developers

Files Modified for DPoP Support

File	Changes
<code>dpop-helper.js</code>	New — DevWeb DPoP proof generation with Node.js crypto

<code>dpop.js</code>	New — VuGen DPoP proof generation with jsrsasign
<code>jsrsasign-vugen.js</code>	New — jsrsasign with exports removed for VuGen JS engine
<code>src/generators/advancedScriptGenerator.js</code>	DPoP detection, proof generation, header replacement (DevWeb)
<code>src/generators/webHttpScriptGenerator.js</code>	DPoP detection, <code>web_js_run</code> generation, header replacement (VuGen)
<code>src/generators/webHttpMandatoryFilesGenerator.js</code>	<code>EnableJsForTransport</code> , <code>ExtraFiles</code> , metadata
<code>src/analyzers/customScriptParser.js</code>	<code>detectDpopUsage()</code> for script-based detection
<code>src/web/public/VuGen-Recorder.html</code>	HAR-based DPoP detection and code generation
<code>src/web/public/VuGen-Script-Studio.html</code>	HAR-based DPoP detection, correlation, code generation
<code>src/web/server.js</code>	Routes to serve <code>dpop-helper.js</code> , <code>dpop.js</code> , <code>jsrsasign-vugen.js</code>
<code>tests/regression.test.js</code>	25 regression tests covering DPoP and non-DPoP scenarios

Isolation Boundaries

DPoP code is guarded by `this.hasDpop` / `S.hasDpop` flags. When no DPoP headers are detected:

- No DPoP-related code is generated
- No DPoP files are copied or included in ZIP
- `EnableJsForTransport` remains 0 (unless JWT is detected)
- All existing functionality is completely unchanged